

Teacher Training Academy Platform – Project Documentation

Project Overview

The **Teacher Training Academy (TTA) Platform** is a scalable digital solution designed to support the structured training of educators, developed under the initiative of **Riyada Trainings**. It combines the robust learning capabilities of **Moodle** with the flexibility and responsiveness of a custom **React.js frontend**, connected via REST APIs. The platform is built to accommodate both institutional clients and individual users, offering seamless multi-tenant management, role-specific interfaces, and e-commerce functionality. The backend extends Moodle's plugin-based architecture to implement advanced learning workflows while ensuring compatibility with future updates.

1. Core Objectives

- Enable **multi-tenant support**, allowing multiple institutions to operate within the same system while keeping their data isolated and secure.
- Deliver a **modern frontend interface** for different user roles including administrators, trainers, and trainees.
- Maintain strict **separation of concerns** by isolating frontend, backend, and API gateway layers.
- Integrate a **B2C-ready e-commerce module** for open course purchases and automatic learner enrollment.
- Ensure compliance with **security best practices**, including token-based authentication, scoped permissions, and input sanitization.

1. Multi-Tenant Capability

- Supports multiple institutions within a single backend instance.
- Each institution is represented in the institutions table using a unique `tenant_id`.
- All key records (users, courses, enrollments) are linked to a `tenant_id` for data isolation.

- Dashboards, permissions, and access are scoped based on the user's tenant.
 - Institutions can operate under:
 - Branded subpaths (e.g., /schoolA)
 - Subdomains (e.g., schoolA.example.com)
-

2. React.js Frontend

- Built using **React.js** for a modern, dynamic user interface.
 - Key Features:
 - **Responsive design** with TailwindCSS or CSS-in-JS.
 - **Role-based layouts** for Admin, Trainer, and Trainee.
 - **Folder structure separation:**
 - /components → UI components
 - /services → API integrations
 - /utils → Business logic/utilities
 - Uses **React Router** for client-side routing.
 - **JWT-based authentication** ensures secure user sessions.
-

3. Moodle Backend (PHP)

- Moodle is the LMS core, extended using **custom plugins**:
 - local, auth, mod, and block plugins for flexibility.
- No modifications to Moodle core—ensures upgrade compatibility.
- Functionality includes:
 - Advanced reporting
 - License control
 - Tenant-specific user/course access
- Integrates Moodle's **REST API and web services** for external access.
- Supports core LMS features:

- Course creation, assignments, grading
 - Certificates, forums, SCORM/xAPI modules
-

4. API Gateway Layer

- Middleware layer developed using **Laravel**
 - Serves as a bridge between the frontend and Moodle backend.
 - Responsibilities:
 - JWT token validation and role checks
 - Input sanitization
 - Middleware-based caching
 - Audit logging and rate limiting
 - Ensures decoupled architecture and protects Moodle internals.
 - Facilitates extension of custom business logic.
-

5. MySQL Database

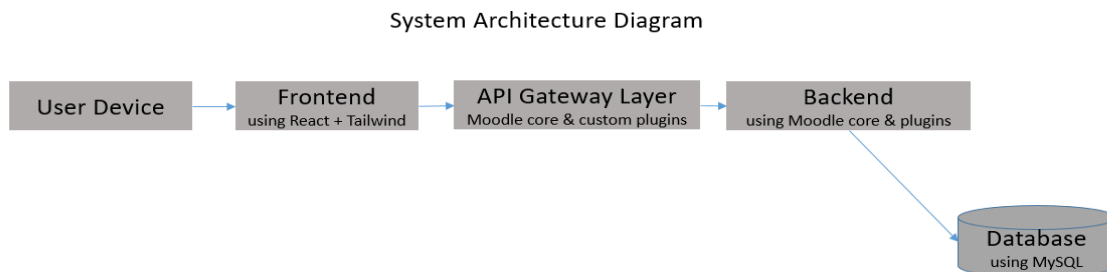
- Centralized database schema with tenant scoping using tenant_id.
 - Key tables: users, courses, enrollments, activities, analytics_logs.
 - **Indexes** applied on tenant_id and foreign keys for performance.
 - **Backup Strategy:**
 - Daily automated MySQL dumps
 - Separate archive per tenant for simplified restoration
-

6. (B2C) Module

- Enables **public course browsing and enrollment** for independent learners.
- Features:
 - Product listing, shopping cart, and secure checkout
- Integrated payment gateways:

- **Stripe, Razorpay, PayPal**
- Backend:
 - Stores orders in orders table and transactions in transactions table
 - Uses **webhooks** to verify successful payments
- On payment success:
 - Users are automatically enrolled into selected courses

2. System Architecture



1. Client Layer (User Devices)

Access Channels:

- Browser-based access across devices: desktop, tablet, and mobile.
- Fully responsive design with Right-to-Left (RTL) support for Arabic.

Features:

- Optimized for modern browsers.
- Integration-ready with video conferencing tools like Zoom and BigBlueButton (BBB) for Virtual Instructor-Led Training (VILT).

2. Frontend Layer (React.js Single Page Application)

Technology Stack:

- Built using React.js with React Router for routing and Context API for global state management.
- Styling powered by TailwindCSS for utility-first responsive design.

Authentication:

- Managed via JWT (JSON Web Tokens), stored securely via HttpOnly cookies or memory-based tokens.

Pages and Navigation:

- Core Screens:
 - Login
 - Dashboard
 - Courses
 - Assessments
 - Reports
 - E-Commerce (B2C)

Components:

- UI Elements: Buttons, Modals, Cards
- Visuals: Charts for analytics and progress tracking

Global State Contexts:

- AuthContext → Manages user authentication and session
- TenantContext → Manages current tenant metadata and state

API Communication:

- All external calls made through Axios to the API Gateway with error handling, loading states, and token headers.

3. API Gateway Layer (Laravel & REST API)

Purpose:

Acts as a middleware layer between the frontend and the Moodle backend, enforcing tenant logic, security, and scalability.

Core Responsibilities:

- Authentication token validation (JWT / OAuth2)
- Tenant-based data scoping
- Rate limiting and API logging
- Unified routing to Moodle's native and extended APIs

Typical Endpoints:

- POST /api/v1/auth/login — User login
- GET /api/v1/courses — Fetch tenant courses
- POST /api/v1/enroll — Enroll user in course

Middleware Pipeline:

1. Token Verifier
2. Tenant Filter
3. Role Checker
4. Rate Limiter
5. Logger

Notes:

- Ensure all incoming requests are scoped by tenant ID.
- Use Laravel policies and middleware to restrict access per role.

4. Backend Layer (Moodle + Custom Plugins)

Core LMS Engine:

- Moodle serves as the learning backend, extended via a custom plugin architecture (no core edits).

Custom Plugin Modules:

- local_tenantmanager → Manages multi-tenant logic (tenant ID isolation)
- local_licensemanager → Manages tenant licensing and limits
- report_analytics → KPI dashboards, usage stats
- mod_certgen → Custom certificate issuing with dynamic templates

Rules and Best Practices:

- All new logic must be implemented as plugins.
 - All access checks must include tenant_id verification.
 - Use Moodle's hooks, events, and capabilities APIs.
-

5. Database Layer (MySQL)

Schema Strategy:

- Single shared schema with tenant_id for tenant separation.
- Core Tables: tenants, users, courses, enrollments, assessments, submissions, certificates, ecommerce_orders

Normalization & Performance:

- Data is normalized up to 3rd Normal Form (3NF).
- Summary tables (e.g., tenant_course_summary) created for faster analytics.

Indexing Strategy:

- Index the following for performance:
 - tenant_id
 - user_id
 - course_id
 - Composite indexes on multi-column filters

Backup Plan:

- Daily tenant-level SQL dumps
- Weekly full encrypted backups to external storage (e.g., Amazon S3)

All access must validate tenant_id

5. Database (MySQL)

- Shared schema with tenant_id on all records
- Stores:

- Tenants, Users, Courses, Enrollments, Assessments, Submissions, Certificates
- E-commerce Orders
- Structure:
 - Normalized to 3NF
 - Analytics use denormalized summary tables (e.g., tenant_course_summary)
- Indexing:
 - tenant_id, user_id, course_id
- Backup:
 - Nightly tenant-wise dumps
 - Weekly off-site (e.g., S3)

3. Key Responsibilities

1. Frontend Layer – React.js SPA

- The frontend is a single-page application (SPA) designed with responsiveness and multilingual support (LTR/RTL) to serve different user roles (Super Admin, Institution Admin, Trainer, Trainee).

Key Responsibilities

- Render dynamic, role-based dashboards
- Display courses, modules, assessments, and certificates
- Handle login/authentication and session state
- Interface securely with backend APIs
- Support English and Arabic with RTL layout

Component Architecture

/src	
/components	→ Reusable UI elements (e.g., Button, Modal)
/pages	→ Views like Dashboard, Courses, Reports
/services	→ Axios API handlers

/contexts	→ AuthContext, TenantContext
/routes	→ Public/Private route configs
/utils	→ Utility and validation logic
/assets	→ Images, icons, translations

State Management

- Local: useState, useReducer
- Global: React Context API
- Planned: Migrate to Redux Toolkit as scale increases

UI Design System

- TailwindCSS for utility-first styling
- Modular CSS where needed
- Responsive layout using Flex/Grid
- Consistent design system across components

Authentication & Security

- Auth via JWT (stored in HttpOnly cookie or memory)
- PrivateRoute component to guard authenticated views
- Token validated before API requests

Primary Modules & Views

- Dashboard (per role)
- Courses Overview & Enrollment
- Assessments & Submissions
- Reports, Certificates, and Progress Tracking
- E-commerce Checkout & Transactions

2. API Gateway Layer – Laravel REST APIs

- This layer serves as a secure middleware between the frontend and the Moodle backend, abstracting complexity and adding control.

Key Responsibilities

- JWT/OAuth2 authentication and token verification
- Tenant-based data scoping
- API rate limiting, logging, and formatting
- Routing to Moodle core and custom plugin APIs

Sample Endpoints

Endpoint	Method	Auth	Description
/api/v1/auth/login	POST		Authenticate user
/api/v1/courses/active	GET		Fetch user’s active courses
/api/v1/enroll	POST		Enroll user in a course
/api/v1/certificates/:id	GET		Download issued certificate

Middleware Stack

- Token Verifier → Tenant Filter → Role Checker → Rate Limiter → Logger

3. Backend – Moodle LMS with Custom Plugins

- Moodle powers the core LMS features, extended with custom plugins for multi-tenancy, licensing, analytics, and certification.

Key Responsibilities

- Manage educational workflows (courses, assessments, grading)
- Enforce role-based access and capabilities
- Handle data scoped per tenant
- Generate and issue dynamic certificates

Key Plugins

Plugin Name	Purpose
local_tenantmanager	Handles tenant isolation and branding
local_licensemanager	Tracks institution license usage
mod_certgen	Generates role-based certificates

report_analytics	Provides tenant-based performance reports
------------------	---

Development Standards

- Use lib.php, renderers, and templates for logic/UI
 - Store custom logic in /classes with PSR-4 autoloading
 - Use access.php and has_capability() for RBAC
 - Avoid core Moodle file modifications
-

4. Database Design

- MySQL shared schema designed for multi-tenant and analytics support.

Core Design

- Each record contains a tenant_id to ensure isolation
- Normalized schema (3NF) for transactional tables
- Denormalized summary tables (e.g., tenant_course_summary) for performance

Practices

- Use FOREIGN KEY with ON DELETE CASCADE
 - Composite indexes (tenant_id, course_id)
 - Avoid nullable fields unless required
 - Enums for fixed-status fields
 - Track created_at, updated_at in all tables
-

5. Multi-Tenancy Strategy

- Shared database model with tenant_id filters on all queries
 - Tenant context derived from JWT token
 - APIs must verify tenant ownership to prevent data leaks
-

6. Authentication & Authorization

- Moodle's OAuth2 plugin extended with JWT issuing logic

- Tokens decoded at API layer and passed to backend
- Roles and permissions enforced using Moodle's `has_capability()`

Role	Permissions Scope
Super Admin (HQ)	All tenants, licenses, settings
Institution Admin	Their own users, courses, reports
Trainer	Course delivery and management
Teacher/Trainee	Participation and learning interactions

7. Deployment Pipeline

- CI/CD via GitHub Actions or GitLab CI
- Branching strategy:
 - main: Production
 - dev: QA/Staging
 - feature/xyz: Feature development
- Run linting, unit tests on PRs
- Auto-deploy to staging, manual deploy to production
- Backup before release with version tags

8. Error Handling & Logging

- React global error boundaries for UI-level failure handling
- API Gateway logs all errors with IP, error code, and timestamp
- Friendly error messages displayed to users
- Logging files stored securely per tenant

9. Local Development Environment

Backend (Moodle)

- Use XAMPP/WAMP stack

- Clone Moodle and custom plugins
- Set up MySQL DB and config.php
- Enable developer debugging in Moodle

Frontend (React)

- Install Node.js (LTS)
- Clone frontend repo, run npm install
- Configure .env for local API
- Start dev server with npm start

10. Performance Optimization

- Enable Moodle cache stores (e.g., memcached)
- Use pagination for API-heavy views
- Lazy-load components and charts
- Compress images (WebP)
- Background jobs for heavy tasks (e.g., report generation)

Code Quality & Best Practices

Modular Structure

- Frontend: Component-driven + route-based organization
- Backend: PSR-4 compliant classes, plugins-only structure

Naming Conventions

Element	Convention	Example
PHP Function	snake_case	get_course_users()
PHP Class	PascalCase	CourseManager
JS Function	camelCase	fetchInstitutions()
CSS Class	kebab-case	main-header-bar

Testing

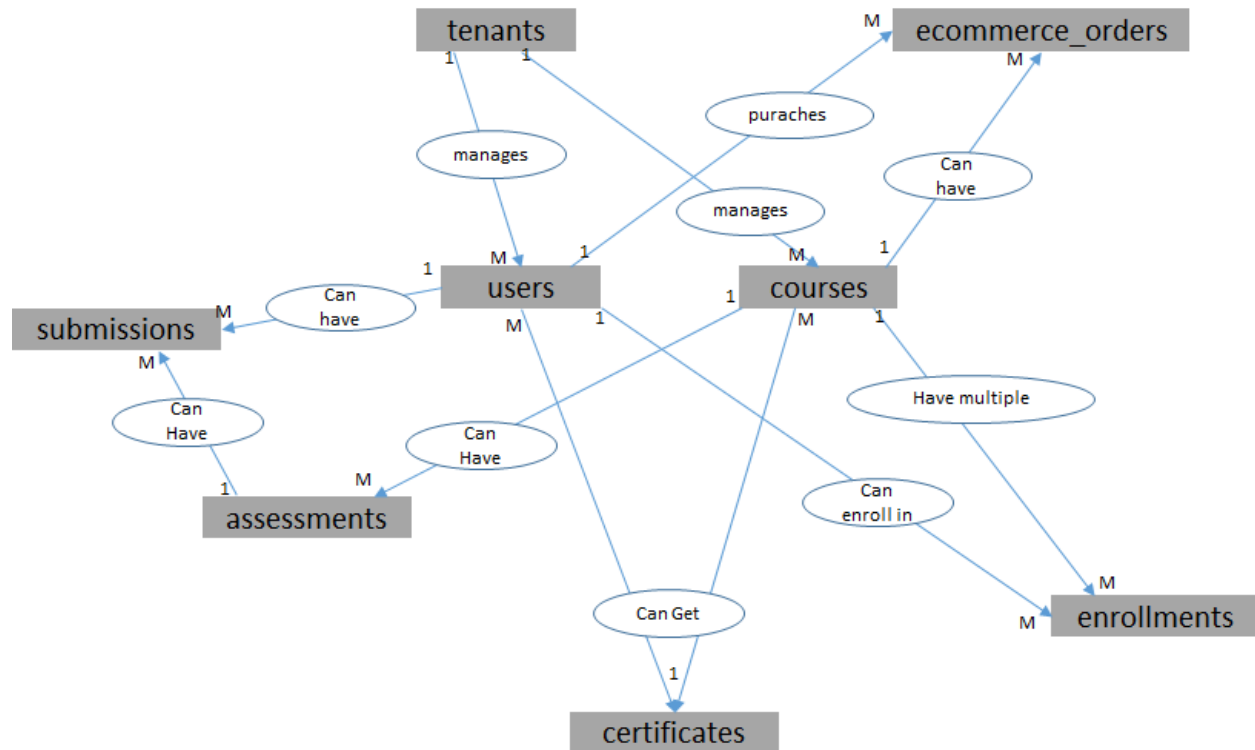
- **Backend:** PHPUnit for Moodle plugin logic
 - **Frontend:** Jest + React Testing Library
 - **UAT:** Performed in staging sandbox before each release
-

Development Kickstart (Suggested Steps)

1. Set up local Moodle LMS and install required plugins
2. Configure Laravel API Gateway and basic endpoints
3. Set up React frontend with base layouts and routes
4. Implement login/auth flow using JWT
5. Pick one module (e.g., course listing) and complete full flow
6. Test across user roles (Admin, Trainer, Trainee)

4. Database Design

This section outlines the core database design used to support both institutional (B2B) and direct learner (B2C) use cases. It focuses on multi-tenancy, data relationships, and modular table design to ensure scalability, data isolation, and performance.



RELATIONSHIP	TYPE	DESCRIPTION
TENANTS → USERS	One-to-Many	A single tenant (e.g., school or organization) manages multiple users.
TENANTS → COURSES	One-to-Many	Each tenant offers its own independent set of courses.
USERS → ENROLLMENTS	One-to-Many	A user can be enrolled in several courses simultaneously.
COURSES → ENROLLMENTS	One-to-Many	Each course can have multiple learners enrolled in it.
COURSES → ASSESSMENTS	One-to-Many	Assessments such as quizzes or reflections are defined under each course.
ASSESSMENTS → SUBMISSIONS	One-to-Many	Multiple users can submit responses to the same assessment.

USERS → SUBMISSIONS	One-to-Many	A user may have many assessment submissions over time.
USERS + COURSES → CERTIFICATES	Many-to-One (Derived)	A certificate is awarded to a user upon completing a specific course.
USERS → ECOMMERCE ORDERS	One-to-Many	A user can make multiple purchases for different courses in the B2C setting.

1) Key Database Table :

Stores metadata for each organization or institution using the platform. Acts as the parent entity in a multi-tenant system.

Field Name	Data Type	Description
id	INT (PK)	Unique tenant ID
name	VARCHAR(255)	Institution or organization name
domain	VARCHAR(255)	Subdomain/path identifier (e.g., tenant1.domain.com)
status	ENUM('active','suspended')	Status of tenant account
created_at	TIMESTAMP	Record creation timestamp
updated_at	TIMESTAMP	Last updated timestamp

2)User Table : Contains login credentials and role-based information for all users (admins, trainers, trainees) within each tenant.

Field Name	Data Type	Description
id	INT (PK)	Unique user ID
tenant_id	INT (FK) → tenants.id	Tenant this user belongs to
username	VARCHAR(100)	Login username
email	VARCHAR(255)	User email
password_hash	TEXT	Hashed password

role	ENUM('super_admin','tenant_admin','trainer','trainee')	User role
status	ENUM('active','inactive')	Account status
created_at	TIMESTAMP	Account creation time
updated_at	TIMESTAMP	Last update

3)Courses: Defines training modules or courses available within a tenant, supporting both B2B and B2C delivery formats.

Field Name	Data Type	Description
id	INT (PK)	Unique course ID
tenant_id	INT (FK) → tenants.id	Associated tenant
title	VARCHAR(255)	Course title
description	TEXT	Course description
level	VARCHAR(50)	Difficulty level (e.g., Beginner)
status	ENUM('draft','published')	Course publication status
is_b2c	BOOLEAN	Available to direct users (B2C)?
created_at	TIMESTAMP	Creation timestamp

4)Enrollments: Tracks which users are enrolled in which courses and monitors their enrollment status and type (self or assigned).

Field Name	Data Type	Description
id	INT (PK)	Enrollment record ID
user_id	INT (FK) → users.id	Enrolled user
course_id	INT (FK) → courses.id	Course enrolled in
enrollment_type	ENUM('self','assigned')	How the enrollment was done
enrolled_at	TIMESTAMP	Enrollment date
status	ENUM('active','completed','dropped')	Progress status

5) Assessment: Stores evaluation components such as quizzes or reflections linked to a course for measuring learning outcomes.

Field Name	Data Type	Description
id	INT (PK)	Assessment ID
course_id	INT (FK) → courses.id	Linked course
title	VARCHAR(255)	Title of the assessment
type	ENUM('quiz','scenario','reflection')	Type of assessment
total_marks	INT	Maximum marks
pass_marks	INT	Minimum passing marks

6) Submissions: Records learners' assessment attempts, scores, and trainer feedback for performance tracking.

Field Name	Data Type	Description
id	INT (PK)	Submission ID
user_id	INT (FK) → users.id	User who submitted
assessment_id	INT (FK) → assessments.id	Assessment attempted
marks_obtained	FLOAT	Score earned
feedback	TEXT	Trainer's feedback
submitted_at	TIMESTAMP	Submission date

7) Certificates : Maintains issued certificates for completed courses along with verification codes and URLs.

Field Name	Data Type	Description
id	INT (PK)	Certificate ID

user_id	INT (FK) → users.id	Certificate holder
course_id	INT (FK) → courses.id	Course completed
issue_date	DATE	Date of issuance
certificate_code	VARCHAR(100)	Unique code
verified_url	VARCHAR(255)	Public verification URL

8) Ecommerce-orders: Logs purchase and payment details for B2C course enrollments made by individual learners.

Field Name	Data Type	Description
id	INT (PK)	Order ID
user_id	INT (FK) → users.id	Purchaser
course_id	INT (FK) → courses.id	Purchased course
amount	DECIMAL(10,2)	Transaction amount
payment_status	ENUM('pending','completed','failed')	Payment state
transaction_id	VARCHAR(255)	Payment gateway response ID
created_at	TIMESTAMP	Order creation date

5) Relational Database Design

This section defines the relational database practices, multi-tenancy strategy, API standards, and authentication mechanisms to ensure secure, maintainable, and analytics-ready development for the TTA platform.

1) Use Foreign Keys with ON DELETE CASCADE

- Ensures automatic cleanup of dependent records when a parent record is deleted.
 - Prevents orphaned data, e.g., assessments and enrollments linked to deleted courses.
-

2)Normalize to 3NF (Third Normal Form)

- Eliminates data redundancy and improves consistency.
- Applies to transactional tables that require frequent updates.

Guidelines:

- Separate entities like course_categories and user_roles into their own tables.
- Replace repeating textual data with foreign key references (e.g., trainer IDs instead of names).

3) Use Denormalized Summary Tables for Analytics

- Enhances performance for dashboard queries by reducing complex joins.
- Summary tables should be refreshed periodically using CRON jobs or triggers.

Example Summary Table: tenant_course_summary

Fields include:

- total_enrollments
- average_score
- completion_rate

Example CRON Logic (Pseudocode):

sql

CopyEdit

For each course in each tenant:

 Calculate total_enrollments, average_score, completion_rate

 Update tenant_course_summary table

4) Integrity Constraints & Optimization Rules

Practice	Benefit
Composite Indexes	Accelerates filtered queries on multi-key lookups
Use ENUMs for status fields	Prevents invalid free-text entries

Filter all queries by tenant_id	Enforces data isolation between tenants
Add audit columns (created_at, updated_at)	Tracks data changes for debugging and auditing
Avoid nullable fields when possible	Promotes strict input validation

5) Multi-Tenancy Implementation

Strategy: Shared Database with Tenant Scoping

- All major tables include a tenant_id field to isolate records.
- API requests must always validate ownership via tenant_id.
- The tenant_id is passed in the JWT and extracted by the API Gateway for context-aware filtering.

6) Authentication & Role-Based Authorization

Auth Mechanism:

- Implements **OAuth2 with JWT** via Moodle and custom token handling.
- The frontend stores the JWT and includes it in the Authorization header for every API call.

Flow:

1. User logs in and receives JWT.
2. Token is passed to the API Gateway.
3. API Gateway decodes the token, validates the user, and applies tenant + role filters.

User Roles:

- super_admin (Riyada platform owner)
- institution_admin (Client organization admin)
- trainer (Course instructor)
- trainee (Learner/student)

Access Control:

- Use Moodle's native has_capability() for enforcing permission checks within plugins.

7) API Architecture

Design Principles:

- Follows **RESTful**, stateless architecture.
- Uses **JSON** for all input and output payloads.
- Secured via **token-based authentication** (JWT).

Standard Endpoints:

Endpoint	Method	Auth Required	Description
/api/login	POST	No	User authentication and token issue
/api/courses	GET	Yes	Retrieve list of courses (tenant-scoped)
/api/users/:id	GET	Yes	Fetch user information
/api/enroll	POST	Yes	Enroll a user into a course

ERD Diagram Location

To support future maintenance and onboarding:

Location for Entity-Relationship Diagrams:

docs/database/erd/

Recommended formats: .png, .svg, or .drawio

8. Multi-Tenancy Approach

Strategy: Shared Database with Tenant ID Scoping

- All primary data tables include a tenant_id column to isolate organizational data.
- All backend queries must include tenant_id filtering.
- The tenant context is passed via JWT token in each API call and validated by the middleware.

Key Rules:

- Avoid cross-tenant access at all times.
- Every API call must verify that the resource belongs to the current authenticated tenant.

- Tenant-based data segregation is enforced across the backend, frontend, and database layers.
-

9. Authentication & Authorization

Mechanism:

- OAuth2 via Moodle's built-in support, extended with JWT tokens for stateless session handling.

Token Flow:

1. The frontend initiates login and retrieves a JWT token.
2. JWT is stored securely (e.g., HttpOnly cookies or in-memory).
3. All API requests attach the token in the Authorization header.
4. The token is decoded and validated in the API Gateway layer.

Access Control:

- Backend roles are enforced using Moodle's native `has_capability()` checks.

Supported Roles:

- `super_admin` → Platform owner (Riyada)
 - `institution_admin` → Manages tenant-specific settings
 - `trainer` → Manages courses, assessments, and feedback
 - `trainee` → End-user learners accessing course content
-

10. API Design

Design Principles:

- Follows **RESTful architecture**
- All endpoints are **stateless**
- Accepts and returns data in **JSON**
- Enforces **token-based authentication**

Example Endpoints:

Endpoint	Method	Auth Required	Description
/api/login	POST	No	User login and JWT issuance
/api/courses	GET	Yes	Returns course list for the tenant
/api/users/:id	GET	Yes	Fetches user details securely
/api/enroll	POST	Yes	Enrolls user in a selected course

11. Frontend Flow (React.js)

11.1. Folder Structure:

bash
CopyEdit
/src
/components → Reusable UI components
/pages → Route-specific screens (Dashboard, Courses)
/services → API integration layer (Axios-based)
/utils → Helper functions and validators
/assets → Static assets (images, icons)

11.2. State Management:

- Uses Context API for global state (AuthContext, TenantContext)
- May upgrade to Redux for large-scale state logic

11.3. Routing:

- Uses React Router v6 for client-side routing
- Defines protected routes to prevent unauthorized access
- Automatically redirects unauthenticated users to login

11.4. Embedding Moodle Content:

- Two options for integration:
 - Embed courses and content using secure iframes
 - Render Moodle-native content via REST APIs using tokens and custom UIs
-

12. Deployment Pipeline

CI/CD Stack:

- Source Control: GitHub or GitLab
- Continuous Integration: GitHub Actions or GitLab CI

Pipeline Stages:

1. Feature branches created for each task
2. Pull Requests must pass linting and unit tests
3. Dev branch auto-deploys to staging server
4. Main branch deploys manually to production

Branching Convention:

- main → Production
 - uat → User acceptance testing
 - dev → Development integration
-

13. Error Handling & Logging

Frontend:

- Uses a global error boundary component in React
- Friendly error messages are shown to users

Backend:

- Logs errors with:
 - Timestamp
 - Error code
 - IP address
 - Affected endpoint

Log Format (Example):

CSS

CopyEdit

[ERROR] [2025-05-29 14:05:11] [IP: 192.168.1.5] [courseController@getCourseDetails] -> Invalid tenant ID

14. Developer Environment Setup Guide

Backend (Moodle):

- Install XAMPP or WAMP
- Clone the Moodle Git repository
- Create a MySQL database and import base schema
- Configure config.php with DB and path settings

Frontend (React.js):

- Install Node.js LTS
 - Run npm install inside the frontend directory
 - Create .env file with API base URL
 - Start development server using npm start
-

15. Performance Optimization Tips

- Enable Moodle caching and performance profiling
 - Paginate large data sets to reduce load
 - Use lazy loading for route-based components
 - Minify and bundle JavaScript and CSS assets
 - Convert and compress images to WebP format for faster delivery
-

16. Steps for Development Team

As the development team, we are coordinating our efforts to move from planning into structured implementation. Below are our immediate next steps:

1. Set Up Local Moodle Instance

- Install and configure Moodle locally for each developer using XAMPP or a similar environment.
- Ensure all necessary plugins and developer tools are installed.
- Sync database structure across team members for consistency.

2. Build API Layer for Custom Calls

- Develop a middleware API layer to interact with Moodle's core functions.
- Use Moodle's **web service API** and extend it where needed with custom plugins.
- Define RESTful endpoints for major actions (e.g., user auth, course retrieval, enrollment).

3. Create Base UI Templates in React

- Set up a new React.js project with Vite or Create React App.
- Establish the design system: fonts, colors, layout components, global state (e.g., Redux, Context API).
- Build base UI components: Header, Footer, Sidebar, Button, Input, and Page Layout wrappers.

4. Integrate Login/Authentication

- Implement **JWT-based authentication**.
- Connect React frontend with the API login endpoint.
- Store and manage JWT tokens securely (e.g., in memory or localStorage).
- Set up route guards and role-based access control (RBAC) for protected pages.

5. Develop One End-to-End Module (Pilot: Course Listing)

- **Back-End:**
 - Create a Moodle web service function or endpoint to fetch course data.
 - Return necessary fields (e.g., title, summary, category, enrollment status).

- **API Layer:**
 - Expose a custom REST endpoint like `/api/courses`.
- **Front-End:**
 - Create a `CourseList` React component.
 - Display list of courses with filters and pagination.
 - Implement loading states and error handling.
- **Testing:**
 - Perform unit testing (Jest) for UI components.
 - Test API responses and error scenarios.
- **Review:**
 - Conduct code reviews and QA as a team.
 - Deploy the module on a shared staging environment.